

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

05 - Árvores e percursos

Prof. Alex Kutzke

Setembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Enunciar a definição recursiva de árvore e aplicar a terminologia;
2. Representar uma árvore binária em Go com uma struct e dois ponteiros;
3. Escrever os percursos em pré-ordem, in-ordem e pós-ordem, e produzir à mão a sequência de visitas de cada um;
4. Escolher o percurso adequado a uma tarefa, justificando pela ordem em que a informação fica disponível;
5. Implementar o percurso em largura com uma fila;
6. Relacionar altura, número de nós e custo de alcançar um nó.

Este conteúdo cai na Prova 1.

Roteiro

Motivação e definição

Terminologia

Representação em Go

Percursos em profundidade

Percurso em largura

Altura e desempenho

Uma hierarquia que você usa todo dia

```
$ tree --charset=ascii 05/demo
```

```
05/demo
```

```
|-- curso
|   |-- ds122
|   |   `-- aula_01.txt
|   `-- ds143
|       |-- aula_01.txt
|       `-- aula_02.txt
`-- leia-me.txt
```

A regra que descreve 05/demo é a mesma que descreve 05/demo/curso
e a mesma que descreve 05/demo/curso/ds143.

O tamanho de cada item

```
$ du -ab 05/demo
```

```
30  05/demo/curso/ds143/aula_01.txt
48  05/demo/curso/ds143/aula_02.txt
78  05/demo/curso/ds143
32  05/demo/curso/ds122/aula_01.txt
32  05/demo/curso/ds122
110 05/demo/curso
46  05/demo/leia-me.txt
156 05/demo
```

- **Ordem:** o diretório aparece depois do que está dentro dele;
- **Aritmética:** $30 + 48 = 78$, $78 + 32 = 110$, $110 + 46 = 156$.

O tamanho do diretório não existe antes do conteúdo

O du não tinha alternativa. O tamanho de um diretório é a soma dos tamanhos do que está dentro, e essa soma só pode ser calculada depois que as partes forem conhecidas.

Ao final da aula essas duas observações voltam com nome: o du percorre o sistema de arquivos em **pós-ordem**, e o tree o percorre em **pré-ordem**.

Você já construiu árvores nesta disciplina

Quick-Union, na aula de análise de algoritmos:

- `id[i]` guardava o **pai** do elemento `i`;
- cada componente conexa era uma árvore, e o conjunto delas uma floresta;
- Find subia da folha até a raiz.

A diferença entre o Quick-Union e o Weighted Quick-Union foi a **altura** dessas árvores: pendurar a menor sob a maior mantém a altura em $O(\log n)$; pendurar sem critério permite que chegue a $O(n)$.

Você já percorreu árvores nesta disciplina

Parser de expressões prefixadas, na aula de recursão:

```
if pos < len(expr) && expr[pos] == '+' {  
    pos++  
    return eval() + eval()  
}
```

- cada chamada de `eval` tratava uma subexpressão;
- as chamadas aninhadas formavam uma árvore;
- ali a árvore **nunca foi construída na memória**: existia apenas como o desenho das chamadas recursivas.

Definição

Uma árvore é um conjunto finito de n nós, tal que, se $n > 0$:

- existe um nó distinguido r , chamado **raiz** da árvore;
- os $n - 1$ nós restantes são divididos em $m \geq 0$ conjuntos disjuntos, cada um deles também uma árvore, chamados **subárvores** de r .

A definição é **recursiva**, e é assim que vamos programar todas as operações de hoje. O caso base é $n = 0$: a **árvore vazia**, que não tem raiz.

Duas consequências da definição

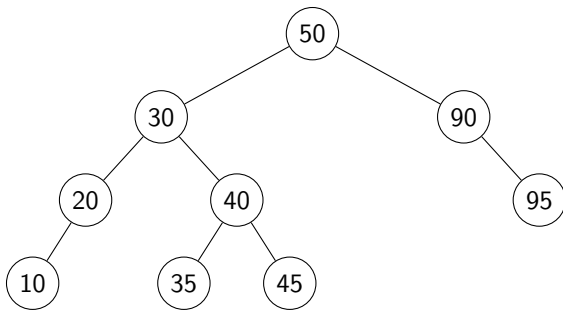
Existe exatamente um caminho da raiz até cada nó

Se houvesse dois, algum nó teria dois pais, e os conjuntos deixariam de ser disjuntos. A estrutura ainda seria útil, mas seria um *grafo*, assunto do final do semestre.

A definição não distingue árvore de subárvore

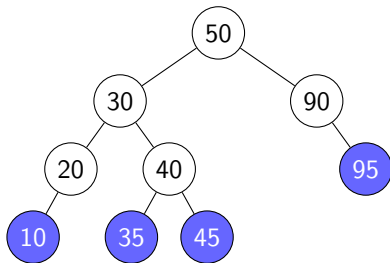
Toda subárvore é uma árvore completa por direito próprio. Uma função que recebe uma árvore pode ser chamada sobre uma subárvore sem nenhuma adaptação.

A árvore de exemplo



Desenhamos a raiz em cima e as folhas embaixo. As ligações apontam sempre do pai para os filhos, e por isso as setas não são desenhadas.

Raiz, pai, filho, folha, nó interno



Raiz o único nó sem pai: 50

Pai de x o nó imediatamente acima: o pai de 35 é 40

Filho de x nó imediatamente abaixo: 20 e 40 são filhos de 30

Folha nó sem filhos: 10, 35, 45, 95

Nó interno nó com pelo menos um filho: 50, 30, 20, 40, 90

Grau, nível e altura

Grau de um nó número de subárvores do nó. Grau de 30 é 2, de 20 é 1, de 10 é 0;

Grau da árvore maior grau entre todos os nós. Aqui, 2;

Nível (profundidade) número de arestas da raiz até o nó. 50 está no nível 0, e 35 no nível 3;

Altura comprimento do caminho mais longo da raiz até uma folha. Aqui, 3;

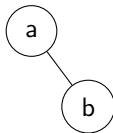
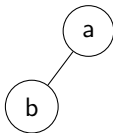
Floresta conjunto de árvores disjuntas, como as componentes do Quick-Union.

Duas convenções desta disciplina

A altura é contada em **arestas**: a árvore com um único nó tem altura 0. A árvore **vazia tem altura -1** , valor escolhido para que $1 + \max(\text{esq}, \text{dir})$ devolva 0 para uma folha.

Árvores binárias

Árvore de grau máximo 2, com uma exigência a mais: cada subárvore é identificada como sendo a da **esquerda** ou a da **direita**. Qualquer uma das duas pode ser vazia.



Iguais enquanto árvores de grau 2, **diferentes** enquanto árvores binárias. A partir da próxima aula a posição do filho carrega informação: à esquerda o que é menor, à direita o que é maior.

A estrutura de um nó

```
type No struct {  
    Info int  
    Esq  *No  
    Dir  *No  
}
```

- O tipo se refere a si mesmo, o que só é possível através de **ponteiros**. Um campo `Esq No` daria um tipo de tamanho infinito, e o compilador recusa (`invalid recursive type No`);
- A árvore inteira é o **ponteiro para o nó raiz**. Um `*No` é ao mesmo tempo um nó e a subárvore que começa nele;
- A árvore vazia é `nil`.

Sem criar vazia, sem liberar

Em C seriam necessárias duas funções a mais:

```
Arvore* cria_arv_vazia (void) { return NULL; }
```

```
Arvore* arv_libera (Arvore* a) {  
    if (a != NULL) {  
        arv_libera(a->esq);  
        arv_libera(a->dir);  
        free(a);  
    }  
    return NULL;  
}
```

Em Go, a árvore vazia é `nil` e o coletor de lixo recolhe os nós que ficam sem referência. Note, de passagem, a ordem das chamadas na `arv_libera`: os filhos antes do `free` do pai.

Duas funções de construção

```
func Constroi(info int, esq, dir *No) *No {  
    return &No{Info: info, Esq: esq, Dir: dir}  
}
```

```
func Folha(info int) *No {  
    return Constroi(info, nil, nil)  
}
```

`&No{...}` cria o nó e devolve o endereço dele. Em Go isso é seguro mesmo com a variável saindo de escopo: o compilador percebe que o endereço escapa da função e aloca o valor onde ele sobreviva.

A árvore de exemplo em uma expressão

```
func ArvoreExemplo() *No {  
    return Constroi(50,  
        Constroi(30,  
            Constroi(20,  
                Folha(10),  
                nil),  
            Constroi(40,  
                Folha(35),  
                Folha(45))),  
        Constroi(90,  
            nil,  
            Folha(95)))  
}
```

O recuo do código reproduz o desenho da árvore.

Procurar um valor

```
func Pertence(a *No, procurado int) bool {  
    if a == nil {  
        return false  
    }  
    if a.Info == procurado {  
        return true  
    }  
    return Pertence(a.Esq, procurado) ||  
        Pertence(a.Dir, procurado)  
}
```

No pior caso, visita todos os n nós, e não há como fazer melhor: nada nesta árvore diz de que lado procurar. Essa é a lacuna que a [árvore binária de busca](#) preenche na próxima aula.

Percorrer e visitar

Percorrer uma árvore é visitar sistematicamente cada um dos seus nós, uma vez cada. **Visitar** um nó é fazer alguma coisa com a informação dele: imprimir, somar, alterar, comparar.

Durante um percurso é comum *passar* por um nó várias vezes sem *visitá-lo*.

Para aplicar um reajuste de 5% em todos os salários guardados na árvore, qualquer ordem serve. Para o *do* da abertura, a ordem era o problema inteiro.

Três ordens, uma diferença

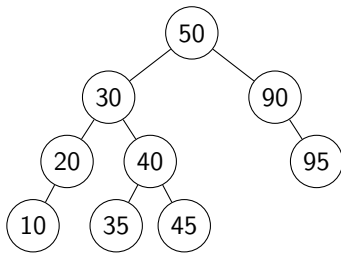
Os três percursos em profundidade descem por um ramo até o fim antes de tentar o ramo seguinte. Diferem apenas em **quando a raiz é visitada**:

Percurso	Ordem	Sigla
Pré-ordem	raiz, esquerda, direita	R, E, D
In-ordem	esquerda, raiz, direita	E, R, D
Pós-ordem	esquerda, direita, raiz	E, D, R

Os três códigos têm as mesmas três linhas, em ordens diferentes.

Pré-ordem (R, E, D)

```
func PreOrdem(a *No) {  
    if a == nil {  
        return  
    }  
    fmt.Print(a.Info, " ")  
    PreOrdem(a.Esq)  
    PreOrdem(a.Dir)  
}
```

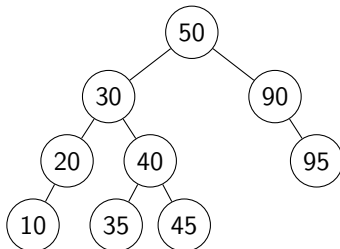


50 30 20 10 40 35 45 90 95

O `if a == nil` é o caso base, e corresponde à árvore vazia da definição. Sem ele o programa quebra na primeira folha.

In-ordem (E, R, D)

```
func InOrdem(a *No) {  
    if a == nil {  
        return  
    }  
    InOrdem(a.Esq)  
    fmt.Print(a.Info, " ")  
    InOrdem(a.Dir)  
}
```

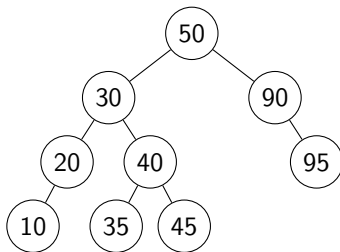


10 20 30 35 40 45 50 90 95

A saída veio em ordem crescente. Isso é propriedade **desta** árvore, montada com os menores à esquerda, e não de toda árvore binária. A próxima aula parte exatamente daí.

Pós-ordem (E, D, R)

```
func PosOrdem(a *No) {  
    if a == nil {  
        return  
    }  
    PosOrdem(a.Esq)  
    PosOrdem(a.Dir)  
    fmt.Print(a.Info, " ")  
}
```



10 20 35 45 40 30 95 90 50

É o percurso do du: 10 e 20 antes de 30, e a raiz 50 no fim de tudo.

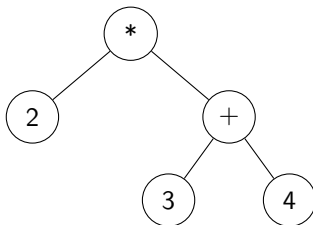
Escolhendo o percurso pela tarefa

Tarefa	Percurso	Por quê
Copiar a árvore preservando a forma	pré-ordem	a raiz precisa existir antes que os filhos sejam pendurados nela
Listar os valores de uma ABB em ordem	in-ordem	é a ordem que a organização da ABB garante
Somar tamanhos, calcular altura, liberar memória	pós-ordem	o resultado do nó depende dos resultados dos filhos
Achar o nó mais próximo da raiz	em largura	os níveis são visitados em ordem crescente de profundidade

A pós-ordem é o caso mais rígido: trocar a ordem das linhas não muda o resultado por gosto, **quebra a função**.

Árvore de expressão

O parser da aula de recursão, agora com a árvore construída de fato:
 $2 \times (3 + 4)$.



Operadores nos nós internos, números nas folhas.

Os três percursos dão as três notações

pré-ordem (prefixa)	:	* 2 + 3 4
in-ordem (sem parênteses)	:	2 * 3 + 4
in-ordem (com parênteses)	:	(2 * (3 + 4))
pós-ordem (pós-fixa)	:	2 3 4 + *
valor	:	14

- A primeira linha é exatamente a entrada que o `prefix_parser.go` lê;
- Prefixa e pós-fixa dispensam parênteses: a posição do operador já determina a estrutura;
- O in-ordem puro perde essa informação. `2 * 3 + 4` lido com a precedência usual vale **10**, e não 14.

Avaliar é um percurso em pós-ordem

```
func Avalia(a *No) int {  
    if a.Esq == nil && a.Dir == nil {  
        valor, _ := strconv.Atoi(a.Info)  
        return valor  
    }  
  
    esq := Avalia(a.Esq)  
    dir := Avalia(a.Dir)  
  
    switch a.Info {  
    case "+":  
        return esq + dir  
    case "*":  
        return esq * dir  
    }  
    return 0  
}
```

Nível a nível

Os três percursos anteriores descem até o fim de um ramo antes de olhar o seguinte. O percurso em largura visita todos os nós do nível 0, depois todos os do nível 1, e assim por diante.

em largura: 50 30 90 20 40 95 10 35 45

pré-ordem : 50 30 20 10 40 35 45 90 95

A pré-ordem alcança o 10, que está no nível 3, antes de alcançar o 90, que está no nível 1.

Por que a recursão não resolve

A recursão dá acesso natural aos **filhos** de quem está sendo visitado. O percurso em largura precisa dos **irmãos**, que estão em outro ramo.

A solução é guardar explicitamente os nós ainda por visitar, em uma **fila**: retira-se um nó da frente, visita-se, e seus filhos entram no fim.

Percurso em largura com uma fila

```
func EmLargura(a *No) {  
    if a == nil {  
        return  
    }  
  
    fila := []*No{a}  
  
    for len(fila) > 0 {  
        atual := fila[0]  
        fila = fila[1:]  
  
        fmt.Print(atual.Info, " ")  
  
        if atual.Esq != nil {  
            fila = append(fila, atual.Esq)  
        }  
        if atual.Dir != nil {  
            fila = append(fila, atual.Dir)  
        }  
    }  
}
```

Trocar a fila por uma pilha devolve a pré-ordem

```

pilha := []*No{a}

for len(pilha) > 0 {
    topo := pilha[len(pilha)-1]
    pilha = pilha[:len(pilha)-1]

    fmt.Print(topo.Info, " ")

    if topo.Dir != nil {
        pilha = append(pilha, topo.Dir)
    }
    if topo.Esq != nil {
        pilha = append(pilha, topo.Esq)
    }
}
```

O filho da direita é empilhado primeiro para que o da esquerda saia antes. É o que a recursão vinha fazendo na pilha de chamadas do programa.

Custo dos percursos

- **Tempo:** $\Theta(n)$ nos quatro. Cada nó é visitado uma vez e cada ponteiro é examinado uma vez;
- **Espaço, em profundidade:** a pilha de chamadas, proporcional à **altura**. $O(\log n)$ na árvore cheia, $O(n)$ na degenerada;
- **Espaço, em largura:** a fila chega a conter um nível inteiro, até $n/2$ nós no último nível de uma árvore cheia.

O percurso em largura é o mesmo **BFS** que aparecerá em grafos, no final do semestre, com a mesma fila e a mesma estrutura de laço.

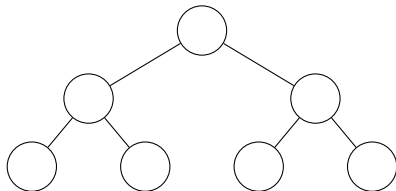
Calcular a altura

```
func Altura(a *No) int {  
    if a == nil {  
        return -1  
    }  
  
    esq := Altura(a.Esq)  
    dir := Altura(a.Dir)  
  
    if esq > dir {  
        return 1 + esq  
    }  
    return 1 + dir  
}
```

O -1 da árvore vazia é o que faz uma folha devolver $1 + (-1) = 0$, sem caso especial.

Árvore cheia

Todo nó interno tem duas subárvores e todas as folhas estão no último nível.



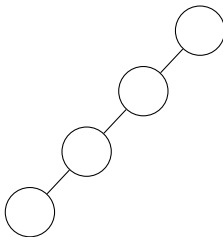
No nível k há 2^k nós. Somando todos os níveis de uma árvore cheia de altura h :

$$n = 2^0 + 2^1 + \dots + 2^h = 2^{h+1} - 1$$

Isolando h : uma árvore cheia com n nós tem altura $h = \log_2(n + 1) - 1$, ou seja, $\Theta(\log n)$.

Árvore degenerada

Um único filho por nó interno, um nó por nível. Com n nós, a altura é $n - 1$.



A forma é a de uma **lista encadeada**, e o desempenho também.

Mesmo número de nós, duas formas

```
$ go run propriedades.go
```

nós		cheia		degenerada
-----	+	-----	+	-----
15		3		14
63		5		62
255		7		254
1023		9		1022

Com 1023 nós, o caminho até o nó mais distante é mais de cem vezes mais longo na árvore degenerada, com a mesma quantidade de informação armazenada.

Altura mínima e altura máxima

Uma árvore binária com n nós tem altura mínima proporcional a $\log_2 n$, no caso da árvore cheia, e altura máxima proporcional a n , no caso da degenerada. A altura indica o esforço para alcançar qualquer nó.

É a mesma diferença entre o Quick-Union e o Weighted Quick-Union, e pela mesma razão. Garantir que a árvore fique perto do primeiro caso, e não do segundo, é o assunto das duas próximas aulas.

De volta ao tree e ao du

- O tree imprime cada diretório **antes** do seu conteúdo: percorre em **pré-ordem**. Quem lê precisa saber em que diretório está antes de ver os arquivos;
- O du imprime cada diretório **depois** do seu conteúdo: percorre em **pós-ordem**. O número do diretório é a soma dos números dos filhos, e não existe antes deles.

O sistema de arquivos não é uma árvore binária: um diretório tem quantos filhos quiser. É o caso geral da definição, com m subárvores. Os percursos continuam valendo, com um laço sobre os filhos no lugar das duas chamadas fixas.

Exercícios

Os exercícios estão no texto da aula. **Não valem nota** e não têm entrega: são a preparação para a aula prática de árvores, essa sim avaliada. O conteúdo cai na Prova 1.

- grau, folhas, altura e nível sobre uma árvore dada;
- as quatro sequências de percurso, à mão;
- ContaNos, Maximo, Espelha, Iguais, NoNoNivel;
- impressão na notação <raiz esq dir>;
- desafio: reconstruir a árvore a partir da pré-ordem e do in-ordem.

Resolva antes da próxima aula, que começa pela árvore binária de busca.